

CSE494

Intelligent NoSQL Databases

UNIT IV — MASTERCLASS

Indexing and Aggregation Framework

Prepared as personal course-preparation material
Session 2026–27

Syllabus Coverage Checklist

Every topic listed in the official CSE494 Unit IV syllabus is mapped below to the exact section that covers it — confirming complete, end-to-end coverage before you begin studying.

Syllabus Topic	Covered In
Indexing basics: what an index is and why it matters	Section 1
Single-field indexes	Section 2
Compound indexes (field order, ESR rule)	Section 3
Text indexes (full-text search)	Section 4
Other index types: multikey, hashed, wildcard (bonus depth)	Section 5
Performance with indexing: write cost, selectivity, cardinality	Section 6
Covered queries and explain("executionStats")	Section 7
Aggregation pipeline concepts: stages, operators	Section 8
Real-world aggregation examples	Section 9
Comic-style analogy for indexing + aggregation	Section 10
Big unified worked example combining indexing + aggregation	Section 11
Full practical walkthroughs (terminal → indexing → aggregation)	Section 12
Practical exercises with solutions	Section 13
Conceptual assessment questions with model answers	Section 14

Contents

- 1. Indexing Fundamentals — Why Indexes Exist
- 2. Single-Field Indexes
- 3. Compound Indexes and the ESR Rule
- 4. Text Indexes — Full-Text Search
- 5. Other Index Types: Multikey, Hashed, Wildcard
- 6. Performance With Indexing — Cost, Selectivity, Cardinality
- 7. Covered Queries and `explain("executionStats")` In Depth
- 8. The Aggregation Pipeline — Core Concepts and Stages
- 9. Real-World Aggregation Examples
- 10. Comic-Style Scenario: The Library Card Catalog & The Assembly Line
- 11. Big Unified Example: Sales Analytics Dashboard (End-to-End)
- 12. Full End-to-End Practical Walkthroughs
- 13. Practical Exercises (with Solutions)
- 14. Self-Assessment: Conceptual Review Questions (with Model Answers)
- 15. Industry Interview & Viva Question Bank
- 16. Frequently Confused Concepts — Quick Clarifications

1. Indexing Fundamentals — Why Indexes Exist

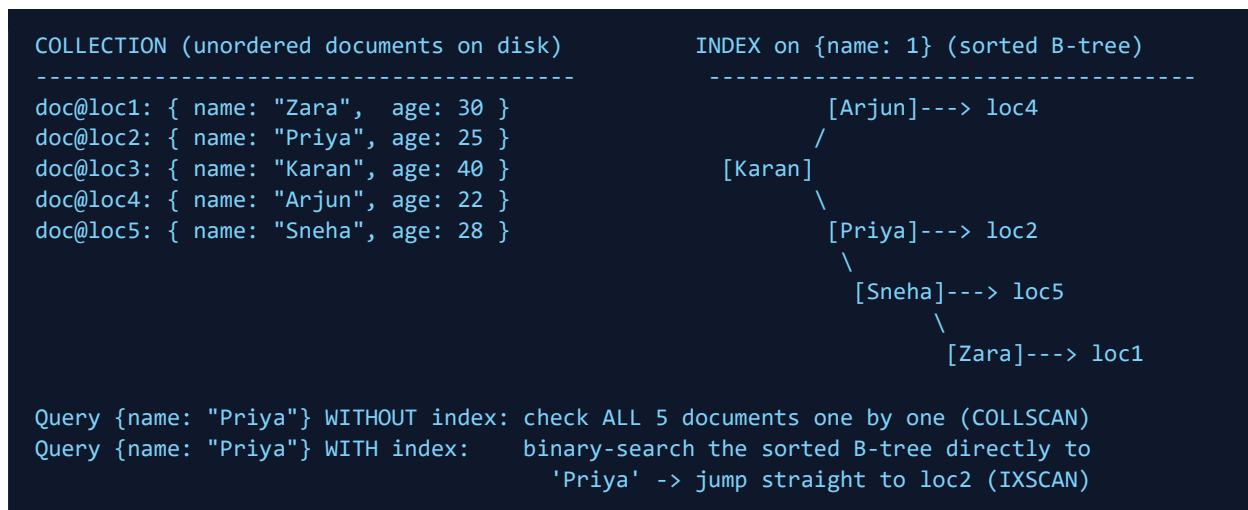
1.1 The Core Problem: Searching Without an Index

Every MongoDB collection is, at its core, an unordered pile of BSON documents stored on disk. When you run a query with no supporting index, MongoDB has exactly one option: walk through EVERY document in the collection, one at a time, checking whether it matches your filter. This is called a COLLECTION SCAN (COLLSCAN), and it is the single most important performance concept in this entire unit.

On a collection with 50 documents, a collection scan is instant — imperceptible to a human. On a collection with 50 million documents, the SAME query could take many seconds, and under concurrent load from thousands of users, it can bring an entire application to its knees. Indexing exists to solve exactly this problem.

1.2 What an Index Actually Is

An index is a separate, ADDITIONAL data structure that MongoDB maintains alongside your actual collection data. It stores a small, sorted subset of field values, each paired with a pointer to the FULL document's location on disk. Internally, MongoDB indexes are implemented as B-trees (specifically, a B-tree variant) — the same fundamental data structure used by virtually every production database engine, including relational ones.



This is directly analogous to the index at the back of a textbook: instead of reading every page to find where 'photosynthesis' is mentioned, you jump to the index, find the sorted entry, and go directly to the listed page number. The book itself (your collection) remains unsorted and unchanged — the index is a separate, purpose-built lookup structure layered on top of it.

1.3 The Default Index: Every Collection Already Has One

As established in Unit II, every MongoDB collection automatically has ONE index from the moment it's created, with zero manual effort: an index on the `_id` field. This is why queries filtering by `_id` are always fast, even on a collection with zero manually-created indexes.

```
// View all indexes currently on a collection
```

```
db.students.getIndexes()

// Even a BRAND NEW, empty collection already has this by default:
// [ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

PRODUCTION NOTE: A very common beginner misconception is assuming ALL fields are automatically indexed just because `_id` is. Every OTHER field you query on is a full collection scan until you explicitly create an index for it — this entire unit is about deciding deliberately WHICH fields deserve that investment.

1.4 The Fundamental Trade-off: Indexes Are Not Free

It's tempting to think 'more indexes = always faster,' but this is wrong. Every index you create must be UPDATED on every insert, update, and delete that touches its indexed field(s) — meaning every index adds a small but real cost to every write operation, plus additional RAM and disk usage to store the index structure itself.

Aspect	Effect of Adding an Index
Reads matching the index	Dramatically faster — from $O(n)$ collection scan to $O(\log n)$ B-tree lookup
Writes touching the indexed field	Slightly slower — the B-tree must be updated to stay in sync with the data
Storage (RAM + disk)	Additional space consumed; ideally the index should fit inside the WiredTiger cache
Query planner complexity	More indexes give the planner more OPTIONS to evaluate, which itself carries a small planning-time cost

NOTE: The entire discipline of indexing, covered across this unit, comes down to ONE central skill: identifying exactly which fields your application ACTUALLY queries on frequently, and creating indexes ONLY for those — not indexing 'just in case.' This mirrors the workload-centric design philosophy from Unit III, applied specifically to indexes.

2. Single-Field Indexes

2.1 Creating a Basic Single-Field Index

```
// Create an ascending index on the 'email' field
db.users.createIndex({ email: 1 })

// Create a descending index on 'createdAt'
db.users.createIndex({ createdAt: -1 })

// View all indexes on a collection, including the default _id index
db.users.getIndexes()
```

The value 1 or -1 specifies SORT DIRECTION within the index (ascending or descending). For a SINGLE-field index, direction almost never matters for query performance — MongoDB can traverse a B-tree in either direction with equal efficiency. Direction becomes meaningful only for COMPOUND indexes involving multiple sort fields, covered in Section 3.

2.2 Unique Indexes — Enforcing Data Integrity

Adding { unique: true } transforms an index from a pure performance tool into a DATA INTEGRITY constraint, rejecting any insert or update that would create a duplicate value for that field — the MongoDB equivalent of a SQL UNIQUE constraint.

```
// Ensure no two users can ever share the same email
db.users.createIndex({ email: 1 }, { unique: true })

// This INSERT will FAIL with a duplicate key error if 'priya@example.com' already
// exists
db.users.insertOne({ email: "priya@example.com", name: "Priya (duplicate attempt)" })
// MongoServerError: E11000 duplicate key error collection: mydb.users index: email_1
```

NOTE: The default `_id` index is, itself, a unique index — this is precisely what guarantees every document's `_id` is unique within its collection without any manual enforcement.

2.3 Sparse Indexes — Handling Optional Fields Efficiently

Because MongoDB documents can have optional fields (Unit III, Section 1), a normal index still creates an entry for every document, even ones missing the field (using a null placeholder). A SPARSE index instead includes ONLY documents where the field actually exists, keeping the index smaller and more efficient for rarely-populated fields.

```
// Only index documents that actually HAVE a 'phoneNumber' field
db.users.createIndex({ phoneNumber: 1 }, { sparse: true })

// Sparse + unique together: enforces uniqueness ONLY among documents that
```

```
// actually have the field -- multiple documents can still have NO phoneNumber
db.users.createIndex({ phoneNumber: 1 }, { sparse: true, unique: true })
```

2.4 Partial Indexes — A More Powerful Alternative to Sparse

A partial index generalizes the sparse concept: instead of only checking 'does the field exist,' it includes documents matching ANY filter expression you specify — strictly more expressive than sparse indexes.

```
// Only index ACTIVE users -- a partial index with an arbitrary condition
db.users.createIndex(
  { email: 1 },
  { partialFilterExpression: { status: "active" } }
)

// A query MUST include the same filter condition (or a subset the planner
// can prove is compatible) for the partial index to actually be used
db.users.find({ email: "priya@example.com", status: "active" })
```

PRODUCTION NOTE: A common mistake: creating a partial index with a condition, then running a query WITHOUT that same condition in the filter. MongoDB's query planner can only use a partial index if it can PROVE the query's results would all satisfy the partial filter — otherwise it silently falls back to a collection scan, with no error raised.

2.5 TTL Indexes — Automatic Document Expiry

Briefly introduced in Unit III, a TTL (Time-To-Live) index is a special single-field index on a Date field that automatically deletes documents once they exceed a configured age — MongoDB's background process checks this roughly every 60 seconds.

```
// Automatically delete session documents 1 hour after their 'createdAt' timestamp
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })
```

NOTE: TTL indexes MUST be single-field indexes on a Date (or array of Dates) field — they cannot be created as part of a compound index. This is a specific, testable constraint worth remembering.

2.6 Dropping an Index

```
// Drop by specifying the exact key pattern
db.users.dropIndex({ phoneNumber: 1 })

// Drop by index name (names are auto-generated, e.g., 'email_1', unless you specify one)
db.users.dropIndex("email_1")

// Drop ALL indexes except the mandatory _id index
db.users.dropIndexes()
```

3. Compound Indexes and the ESR Rule

3.1 What Is a Compound Index?

A compound index covers MULTIPLE fields in a single, deliberately-ordered B-tree structure. It's created exactly like a single-field index, just listing more than one field.

```
db.orders.createIndex({ customerId: 1, orderDate: -1 })
```

Think of a compound index like a phone book sorted first by LAST NAME, then by FIRST NAME within each last name. You can efficiently find 'all Smiths,' or 'the Smith named John' — but you CANNOT efficiently find 'everyone named John' regardless of last name, because the phone book isn't primarily sorted by first name.

3.2 The Prefix Rule

A compound index on { a: 1, b: 1, c: 1 } can efficiently support queries filtering on ANY PREFIX of that field list: { a }, { a, b }, or { a, b, c } — but NOT on { b } alone, or { c } alone, or { b, c } without a.

```
db.orders.createIndex({ customerId: 1, status: 1, orderDate: -1 })

// EFFICIENT -- uses the index (matches a valid prefix)
db.orders.find({ customerId: ObjectId("...") })
db.orders.find({ customerId: ObjectId("..."), status: "shipped" })
db.orders.find({ customerId: ObjectId("..."), status: "shipped" }).sort({ orderDate: -1 })

// INEFFICIENT -- does NOT use this index (customerId, the leading field, is missing)
db.orders.find({ status: "shipped" })
db.orders.find({ orderDate: { $gt: ISODate("2026-01-01") } })
```

PRODUCTION NOTE: This is one of the most heavily tested concepts in MongoDB indexing: the ORDER of fields in a compound index is not arbitrary — it fundamentally determines which queries can and cannot use it. Getting the order wrong means creating an index that silently doesn't help your actual queries.

3.3 The ESR Rule — Equality, Sort, Range

MongoDB's official, widely-cited guideline for ordering fields in a compound index is the ESR rule: place EQUALITY filter fields first, SORT fields second, and RANGE filter fields last.

Position	Field Type	Why This Position
E — Equality	Fields queried with exact-match conditions (field: value or \$eq)	Equality matches narrow the B-tree traversal to a single exact point FIRST, most efficiently reducing the search space before anything else happens.
S — Sort	Fields used in .sort()	Placing sort fields right after equality fields lets MongoDB read results DIRECTLY off the index in already-sorted order, avoiding an expensive separate in-memory sort step.

Position	Field Type	Why This Position
R — Range	Fields queried with range conditions (\$gt, \$lt, \$gte, \$lte, \$ne, \$in)	Range conditions scan a CONTIGUOUS span of the B-tree rather than a single point, so they must come last — putting a range field before a sort field would break the index's ability to serve pre-sorted results.

```
// Query pattern: filter by exact 'status', sort by 'orderDate' descending,
// filter 'totalAmount' within a range
db.orders.find(
  status: "shipped",                      // E - Equality
  totalAmount: { $gt: 1000, $lt: 50000 } // R - Range
).sort({ orderDate: -1 })                // S - Sort

// Applying ESR: Equality (status) -> Sort (orderDate) -> Range (totalAmount)
db.orders.createIndex({ status: 1, orderDate: -1, totalAmount: 1 })
```

NOTE: Common exam trap: students often order fields alphabetically, or in the order they appear in the query, rather than applying ESR. Always explicitly classify each field as E, S, or R first, THEN order the index — do not guess from the query's visual field order.

3.4 Verifying a Compound Index With explain()

```
// BEFORE the ESR-ordered index exists
db.orders.find({ status: "shipped", totalAmount: { $gt: 1000 } })
  .sort({ orderDate: -1 }).explain("executionStats")
// winningPlan.stage: "COLLSCAN" or an inefficient IXSCAN + in-memory SORT stage

db.orders.createIndex({ status: 1, orderDate: -1, totalAmount: 1 })

// AFTER
db.orders.find({ status: "shipped", totalAmount: { $gt: 1000 } })
  .sort({ orderDate: -1 }).explain("executionStats")
// winningPlan.stage: "IXSCAN" -- with NO separate in-memory SORT stage in the plan,
// because the index already returns orderDate in sorted order
```

3.5 Compound Indexes and Multikey Behavior

If ONE of the fields in a compound index is an array field, the index becomes a MULTIKEY index (fully covered in Section 5.1). An important restriction: a compound index can have AT MOST ONE array field — MongoDB disallows compound indexes across two DIFFERENT array fields, because the combinatorial explosion of index entries (every element of array A paired with every element of array B) would be impractical.

```
// ALLOWED -- only one array field (tags) in the compound index
db.products.createIndex({ category: 1, tags: 1 })

// DISALLOWED -- 'tags' and 'sizes' are BOTH arrays -- this throws an error
// db.products.createIndex({ tags: 1, sizes: 1 })
// MongoServerError: cannot index parallel arrays
```


4. Text Indexes — Full-Text Search

4.1 Why Regular Indexes Can't Do Full-Text Search

A normal B-tree index is excellent for exact matches and range queries, but terrible for searching **WITHIN** text — e.g., finding all blog posts containing the word 'mongodb' anywhere in a long article body. A regex query like { content: /mongodb/i } CANNOT use a normal index efficiently (unless it's a simple prefix match anchored with ^), forcing a full collection scan and manual text scanning on every document.

A text index solves this by pre-processing string content into a searchable structure: it tokenizes text into individual words, removes common 'stop words' (like 'the', 'a', 'is'), applies stemming (so 'running' and 'run' are treated as related), and builds an inverted index mapping each word to the documents containing it.

4.2 Creating a Text Index

```
// Text index on a single field
db.articles.createIndex({ content: "text" })

// Text index spanning MULTIPLE fields -- searches across both simultaneously
db.articles.createIndex({ title: "text", content: "text" })

// Weighted text index -- matches in 'title' count MORE toward relevance
// scoring than matches in 'content' (useful when a title match is a
// stronger signal of relevance than a body match)
db.articles.createIndex(
  { title: "text", content: "text" },
  { weights: { title: 10, content: 1 } }
)
```

PRODUCTION NOTE: A collection can have AT MOST ONE text index (though that single text index can span multiple fields, as shown above). This is a hard MongoDB restriction, unlike regular indexes where you can create as many as you need.

4.3 Querying With \$text and \$search

```
// Basic text search -- finds documents containing ANY of the given words
db.articles.find({ $text: { $search: "mongodb performance" } })

// Phrase search -- wrap in escaped double quotes to require the EXACT phrase
db.articles.find({ $text: { $search: "\"query optimization\"" } })

// Exclude a word using a minus prefix
db.articles.find({ $text: { $search: "mongodb -sql" } }) // mentions mongodb, NOT
sql

// Retrieve and sort by RELEVANCE SCORE (how well each document matches)
db.articles.find(
```

```
{ $text: { $search: "mongodb performance" } },
{ score: { $meta: "textScore" } }
).sort({ score: { $meta: "textScore" } })
```

4.4 Text Index Limitations and When to Use a Dedicated Search Engine Instead

MongoDB's text index is genuinely useful for basic full-text search needs, but it is intentionally simpler than a dedicated search engine (like Elasticsearch or MongoDB Atlas Search, which is built on Lucene).

Capability	MongoDB \$text Index	Dedicated Search Engine (e.g., Atlas Search)
Basic keyword matching	Yes	Yes
Relevance scoring	Basic (\$meta: 'textScore')	Advanced, highly tunable relevance algorithms
Typo tolerance / fuzzy matching	No	Yes
Faceted search, autocomplete	No	Yes
Multiple simultaneous text indexes per collection	No (max one)	Not a limiting factor

NOTE: Practical guidance: for simple 'search this field for a keyword' needs within an application already built on MongoDB, a \$text index is often sufficient and requires no additional infrastructure. For a genuine search-heavy product feature (e.g., an e-commerce search bar with typo tolerance and autocomplete), a dedicated search layer is the professional choice — knowing this distinction, and not over-relying on \$text for search-heavy UX, is itself a valuable piece of engineering judgment.

5. Other Index Types: Multikey, Hashed, Wildcard

Beyond single-field, compound, and text indexes, MongoDB offers several specialized index types worth knowing for a complete picture of the indexing toolkit — and because they appear frequently in real-world interview questions.

5.1 Multikey Indexes — Indexing Array Fields

When you create an index on a field that holds an ARRAY, MongoDB automatically creates a multikey index — internally, one index entry is created for EACH element of the array, all pointing back to the same parent document.

```
db.products.insertOne({ name: "T-Shirt", tags: ["apparel", "summer", "cotton"] })
```

```
// Creating a normal index on an array field automatically becomes multikey
db.products.createIndex({ tags: 1 })
```

```
// A query matching ANY element of the array uses the multikey index efficiently
db.products.find({ tags: "summer" })
```

```
Document: { _id: 1, tags: ["apparel", "summer", "cotton"] }
```

```
Multikey index on 'tags' creates THREE separate entries,
all pointing to the SAME document:
```

```
[apparel] ---> _id: 1
[cotton]   ---> _id: 1
[summer]   ---> _id: 1
```

NOTE: You don't explicitly request a 'multikey' index — MongoDB detects an array field automatically and builds it that way. The only manual consideration is the earlier restriction (Section 3.5): a compound index can include at most ONE array field.

5.2 Hashed Indexes

A hashed index stores the HASH of a field's value rather than the value itself, distributing entries evenly and unpredictably across the index's range. This makes hashed indexes unsuitable for range queries (a hash destroys ordering information) but ideal for one specific purpose: hashed SHARDING (a Unit I/advanced topic), where an even distribution of data across shards is more important than range-query support.

```
// Create a hashed index -- commonly used as a shard key
db.users.createIndex({ userId: "hashed" })
```

```
// GOOD use case: exact equality lookups still work efficiently
db.users.find({ userId: "u12345" })
```

```
// POOR use case: range queries CANNOT use a hashed index efficiently
```

```
// db.users.find({ userId: { $gt: "u10000" } }) -- falls back to COLLSCAN
```

5.3 Wildcard Indexes

A wildcard index lets you index an UNKNOWN or highly variable set of fields across a collection with a genuinely unpredictable schema — directly useful for MongoDB's schema-flexible nature (Unit III). Instead of naming specific fields, you use the `$**` wildcard.

```
// Index EVERY field in every document -- useful for collections with
// highly unpredictable, per-document-varying structure
db.userGeneratedContent.createIndex({ "$**": 1 })

// Index every field WITHIN a specific sub-document only
db.products.createIndex({ "attributes.$**": 1 })

// This directly supports the Attribute Pattern from Unit III --
// a wildcard index can cover querying by ANY attribute key without
// needing to know the attribute names in advance
db.products.find({ "attributes.color": "Red" })
```

PRODUCTION NOTE: Wildcard indexes are powerful but come with real costs: they can be significantly larger than a targeted index (since they index MANY fields), and MongoDB explicitly recommends using them only when you genuinely cannot predict which fields will be queried — for known, stable query patterns, a specific targeted index (Sections 2–3) is almost always more efficient.

5.4 Quick-Reference Summary Table

Index Type	Best For	Key Restriction
Single-field	One field, simple equality/range queries	None
Compound	Multiple fields queried together, following ESR	At most one array field per compound index
Text	Full-text search across string fields	At most ONE text index per collection
Multikey	Automatic — any index on an array field	Compound multikey limited to one array field
Hashed	Even data distribution, commonly for sharding	Cannot efficiently serve range queries
Wildcard	Unpredictable/highly variable schemas	Larger index size; avoid when specific fields are known

6. Performance With Indexing — Cost, Selectivity, Cardinality

6.1 The Write-Performance Cost, Quantified

Every index adds overhead to writes. To make this concrete rather than abstract: inserting one document into a collection with 5 indexes requires MongoDB to perform UP TO 6 separate B-tree insertions internally (1 for the actual document location tracking plus updates to each of the 5 indexes) — versus just 1 with zero indexes. This overhead is usually small per-operation, but it compounds significantly under high write throughput.

```
// Measuring the practical impact -- insert 10,000 documents with NO indexes
db.testNoIndex.drop()
let start = Date.now()
let batch = []
for (let i = 0; i < 10000; i++) { batch.push({ n: i, val: "data" + i }) }
db.testNoIndex.insertMany(batch)
print("No index insert time (ms):", Date.now() - start)

// Now with THREE indexes already in place before the same insert workload
db.testWithIndex.drop()
db.testWithIndex.createIndex({ n: 1 })
db.testWithIndex.createIndex({ val: 1 })
db.testWithIndex.createIndex({ n: 1, val: -1 })
start = Date.now()
db.testWithIndex.insertMany(batch)
print("3-index insert time (ms):", Date.now() - start)

// The indexed version will consistently take measurably longer --
// this is the concrete, observable cost discussed abstractly until now
```

NOTE: This is a genuinely useful exercise to run yourself in mongosh — seeing the timing difference firsthand builds much stronger intuition than reading about it abstractly, and is exactly the kind of hands-on verification a production engineer would do before adding indexes to a write-heavy collection.

6.2 Selectivity — Why SOME Fields Make Better Indexes Than Others

Selectivity measures how effectively a field's value narrows down a search — a HIGHLY selective field has many distinct values, so matching on it eliminates most other documents immediately. A LOW selectivity field has few distinct values, so matching on it barely narrows the search at all.

Field Example	Selectivity	Indexing Verdict
email (near-unique per user)	Very high — thousands of distinct values, ~1 document per value	Excellent index candidate
status (e.g., only 'active'/'inactive')	Very low — only 2 distinct values across millions of documents	Poor STANDALONE index candidate — matching 'active' might still return 90% of the collection

Field Example	Selectivity	Indexing Verdict
gender (e.g., 'M'/'F'/'Other')	Very low — 3 distinct values	Poor standalone index candidate, same reasoning
userId (unique per user)	Maximum — every value is unique	Excellent index candidate (often the PRIMARY lookup key)

The intuition: an index on a low-selectivity field like status still requires MongoDB to fetch and examine a LARGE fraction of the matching index entries — barely better than scanning the whole collection, while still paying the full write-cost overhead of maintaining that index.

PRODUCTION NOTE: This does NOT mean low-selectivity fields should never be indexed — it means they're rarely worth indexing ALONE. A low-selectivity field like status is often a GREAT candidate as the leading equality field in a COMPOUND index (Section 3), where it's combined with a more selective field to narrow results efficiently together.

6.3 Cardinality vs. Selectivity — A Precise Distinction

These two terms are often used loosely and interchangeably, but a precise understanding helps in interviews. Cardinality is the RAW COUNT of distinct values a field has across the collection. Selectivity is a RATIO — how much a specific query VALUE for that field narrows the result set, relative to the collection's total size.

```
// A field can have HIGH cardinality overall, but a specific query value
// can still have LOW selectivity for that particular value -- e.g.:

// 'city' has high cardinality (thousands of distinct cities globally)
// BUT querying { city: "Mumbai" } on an India-focused user base might
// still match millions of documents -- LOW selectivity for THAT value,
// even though the FIELD overall has high cardinality
```

NOTE: Exam-safe distinction: cardinality is a property of the FIELD (how many distinct values exist); selectivity is a property of a specific QUERY against that field (what fraction of documents a particular value matches). High cardinality usually correlates with high selectivity, but not always — this India/Mumbai example is exactly the kind of nuance a strong answer would include.

6.4 Index Intersection — When MongoDB Combines Multiple Indexes

In some cases, MongoDB's query planner can use TWO SEPARATE single-field indexes together for one query, intersecting their results — though this is generally less efficient than a single well-designed compound index covering the same query.

```
db.orders.createIndex({ status: 1 })
db.orders.createIndex({ region: 1 })

// The planner MIGHT intersect these two separate indexes for this query...
db.orders.find({ status: "shipped", region: "North" })

// ...but a purpose-built compound index is almost always faster and
```



```
// is the recommended, deliberate choice rather than relying on intersection  
db.orders.createIndex({ status: 1, region: 1 })
```

NOTE: Interview-relevant takeaway: index intersection exists as a query planner capability, but it should be understood as a FALLBACK the planner might use, not a design STRATEGY you should rely on. A well-designed compound index, chosen deliberately per your actual query patterns (following ESR), is always the professional recommendation over hoping the planner intersects several single-field indexes efficiently.

7. Covered Queries and explain("executionStats") In Depth

7.1 What Is a Covered Query?

A covered query is the FASTEST possible category of MongoDB query: one where EVERY field requested (both in the filter AND the projection) is present in the index itself, meaning MongoDB never needs to fetch the actual document from disk/cache at all — it answers the query entirely from the index's own B-tree data.

```
db.students.createIndex({ email: 1, name: 1 })

// COVERED query -- filter field (email) AND projected fields (name)
// are BOTH fully present in the index; _id is explicitly excluded
db.students.find(
  { email: "priya@example.com" },
  { name: 1, _id: 0 }
)

// NOT covered -- 'cgpa' is projected but NOT part of the index,
// forcing MongoDB to fetch the full document anyway
db.students.find(
  { email: "priya@example.com" },
  { name: 1, cgpa: 1, _id: 0 }
)
```

PRODUCTION NOTE: A subtle but critical rule: if `_id` is NOT explicitly excluded (`_id: 0`) in the projection, the query CANNOT be covered — because `_id` is always returned by default, and unless `_id` itself is part of the index, MongoDB must fetch the full document just to retrieve it.

7.2 Identifying a Covered Query in explain() Output

```
db.students.find(
  { email: "priya@example.com" },
  { name: 1, _id: 0 }
).explain("executionStats")
```

The key field to check is `totalDocsExamined` — for a truly covered query, this value is 0, because MongoDB never touched the actual document data, satisfying the entire query from the index alone.

executionStats field	Value for a COVERED query	Value for a normal INDEXED (but not covered) query
<code>totalDocsExamined</code>	0 — no document data fetched at all	Equal to (roughly) <code>nReturned</code> — documents ARE fetched, just efficiently via the index
<code>totalKeysExamined</code>	Equal to <code>nReturned</code> — index entries scanned	Equal to <code>nReturned</code> — same
<code>executionStages.stage</code>	Shows <code>PROJECTION_COVERED</code> as part of the plan	Shows a normal <code>FETCH</code> stage pulling full documents

7.3 The Full explain() Output Structure

explain() supports three verbosity modes, each revealing progressively more detail:

Mode	What It Shows	When to Use
"queryPlanner" (default if no argument)	Only the CHOSEN plan and rejected alternative plans — no actual execution happens	Quick check of which plan WOULD be chosen, without running the query
"executionStats"	Actually RUNS the query and reports real timing, docs examined, keys examined	The standard mode for diagnosing real query performance (used throughout this course)
"allPlansExecution"	Runs the query AND all candidate plans the planner considered, with stats for each	Deep debugging when you need to understand WHY the planner picked a particular plan over another

```
// The three verbosity levels in practice
db.students.find({ branch: "CSE" }).explain("queryPlanner")
db.students.find({ branch: "CSE" }).explain("executionStats")
db.students.find({ branch: "CSE" }).explain("allPlansExecution")
```

7.4 Reading executionStats Field by Field

```
{
  executionStats: {
    executionSuccess: true,
    nReturned: 42,                // documents actually returned to the client
    executionTimeMillis: 3,       // total wall-clock time for the query
    totalKeysExamined: 42,        // index B-tree entries scanned
    totalDocsExamined: 42,        // actual documents fetched from storage
    executionStages: {
      stage: "FETCH",            // outer stage: fetching full documents
      inputStage: {
        stage: "IXSCAN",         // inner stage: index scan supplied the keys
        indexName: "branch_1",
        direction: "forward"
      }
    }
  }
}
```

A healthy, well-optimized query has totalDocsExamined close to nReturned (ideally equal), and totalKeysExamined close to nReturned as well. A LARGE gap between totalDocsExamined and nReturned is the single most important red flag to look for — it means MongoDB examined far more data than it actually needed to return, almost always signaling a missing or poorly-ordered index.

7.5 A Complete Before/After Diagnostic Walkthrough

```
// STEP 1: baseline -- no index, observe the problem
db.orders.find({ status: "pending", region: "North" }).explain("executionStats")
// stage: COLLSCAN
// totalDocsExamined: 500000 (the ENTIRE collection)
// nReturned: 340
// executionTimeMillis: 210

// STEP 2: diagnose -- massive gap between totalDocsExamined (500000) and
// nReturned (340) confirms a missing index is the root cause

// STEP 3: apply ESR (status=Equality, region=Equality -- both equality, order
// between them matters less, but put the MORE selective field first)
db.orders.createIndex({ region: 1, status: 1 })

// STEP 4: verify the fix
db.orders.find({ status: "pending", region: "North" }).explain("executionStats")
// stage: IXSCAN -> FETCH
// totalDocsExamined: 340 (matches nReturned almost exactly!)
// executionTimeMillis: 2
```

NOTE: This four-step workflow — baseline, diagnose via the totalDocsExamined-vs-nReturned gap, apply an ESR-ordered index, verify with explain() again — is the single most practically valuable skill in this entire unit, and is very likely to appear directly as a hands-on task in an exam, lab practical, or technical interview.

8. The Aggregation Pipeline — Core Concepts and Stages

8.1 What Is the Aggregation Pipeline?

`find()` is designed for retrieving documents largely as-is — filtering, projecting, sorting. But real applications constantly need to TRANSFORM and SUMMARIZE data: total sales per region, average rating per product, the top 10 most active users. This is what the Aggregation Framework is for — a pipeline of sequential processing STAGES, where each stage takes the output of the previous stage as its input, progressively transforming documents until the final result emerges.

```
Collection      Stage 1      Stage 2      Stage 3      Result
(raw documents) --> $match      --> $group      --> $sort      --> (final output)
                  (filter      (aggregate   (order
                  down to     into groups,   results)
                  relevant docs) compute totals)
```

Just like a factory ASSEMBLY LINE: each station performs ONE transformation, passing its output to the next station, until a finished product emerges.

```
db.<collection>.aggregate([
  { $stage1: { ... } },
  { $stage2: { ... } },
  { $stage3: { ... } }
])
```

8.2 \$match — Filtering Documents

`$match` filters documents, using the exact SAME query syntax as `find()`'s filter argument. It is almost always the FIRST stage in a well-designed pipeline, since filtering early reduces the volume of data every subsequent stage has to process.

```
db.orders.aggregate([
  { $match: { status: "completed", orderDate: { $gte: ISODate("2026-01-01") } } }
])
```

NOTE: Critically, `$match` stages placed EARLY in a pipeline (especially as the very first stage) CAN use indexes exactly like a normal `find()` query — this is one of the most important aggregation performance principles, covered further in Section 8.9.

8.3 \$group — Aggregating Into Summarized Groups

`$group` is the heart of the aggregation framework — it groups documents by a specified key (`_id` within the stage) and computes summary values across each group using ACCUMULATOR operators.

```
db.orders.aggregate([
  { $group: {
    _id: "$customerId",           // group by this field
```

```

    totalSpent: { $sum: "$totalAmount" },
    orderCount: { $sum: 1 },           // counting documents per group
    avgOrderValue: { $avg: "$totalAmount" },
    largestOrder: { $max: "$totalAmount" }
  } }
])

```

Accumulator	Purpose	Example
\$sum	Total of a numeric field (or count documents via \$sum: 1)	{ \$sum: "\$amount" }
\$avg	Average of a numeric field across the group	{ \$avg: "\$rating" }
\$min / \$max	Smallest / largest value in the group	{ \$max: "\$price" }
\$push	Collect values from the group into an array (duplicates allowed)	{ \$push: "\$productName" }
\$addToSet	Collect UNIQUE values from the group into an array	{ \$addToSet: "\$category" }
\$first / \$last	The first/last value encountered in the group (order-dependent — pair with \$sort first)	{ \$first: "\$status" }

Grouping by null aggregates the ENTIRE collection into a single group — useful for computing a grand total across all documents:

```

db.orders.aggregate([
  { $group: {
    _id: null,                // a single group covering ALL documents
    grandTotal: { $sum: "$totalAmount" },
    totalOrders: { $sum: 1 }
  } }
])

```

8.4 \$project — Reshaping Documents

\$project controls exactly which fields appear in the output, similar to find()'s projection but far more powerful — it can also COMPUTE new fields using expressions.

```

db.orders.aggregate([
  { $project: {
    _id: 0,
    customerId: 1,
    total: "$totalAmount",      // rename a field
    itemCount: { $size: "$items" }, // compute the array length
    isLargeOrder: { $gt: ["$totalAmount", 10000] } // compute a boolean
  } }
])

```

8.5 \$sort, \$limit, \$skip — Ordering and Pagination

```
db.orders.aggregate([
  { $match: { status: "completed" } },
  { $sort: { totalAmount: -1 } }, // highest-value orders first
  { $skip: 0 },
  { $limit: 10 } // top 10 orders
])
```

NOTE: Just like with `find()` cursor methods (Unit II), placing `$sort` BEFORE `$limit` is essential — sorting only the already-limited results would produce the wrong answer entirely (an arbitrary 10 documents, sorted, rather than the TRUE top 10).

8.6 \$unwind — Flattening Arrays

`$unwind` deconstructs an array field, producing ONE separate output document for EACH element of the array — essential when you need to aggregate on individual array elements rather than the array as a whole.

```
// A single order document with an 'items' array
// { _id: 1, customer: "Priya", items: [
//   { product: "Laptop", price: 55000 }, { product: "Mouse", price: 599 } ] }

db.orders.aggregate([
  { $unwind: "$items" }
])

// Produces TWO separate documents:
// { _id: 1, customer: "Priya", items: { product: "Laptop", price: 55000 } }
// { _id: 1, customer: "Priya", items: { product: "Mouse", price: 599 } }

// Now the total revenue PER PRODUCT (across all orders) can be computed --
// impossible to do directly on the un-flattened array
db.orders.aggregate([
  { $unwind: "$items" },
  { $group: {
    _id: "$items.product",
    totalRevenue: { $sum: "$items.price" }
  } }
])
```

8.7 \$lookup — Joining Collections

Introduced in Unit III, `$lookup` performs a left-outer-join-style operation against another collection at query time. Full syntax recap:

```
db.orders.aggregate([
  { $lookup: {
    from: "customers",
    localField: "customerId",
    foreignField: "_id",
```

```

    as: "customerInfo"
  } },
  { $unwind: "$customerInfo" } // flatten the resulting array into a single object
])

```

An advanced form uses a full sub-pipeline instead of simple field matching, enabling filtering and shaping of the joined data BEFORE it's attached:

```

db.orders.aggregate([
  { $lookup: {
    from: "customers",
    let: { custId: "$customerId" },
    pipeline: [
      { $match: { $expr: { $eq: ["$_id", "$$custId"] } } },
      { $project: { name: 1, email: 1, _id: 0 } } // only bring back these fields
    ],
    as: "customerInfo"
  } }
])

```

8.8 \$addFields — Adding Computed Fields Without Dropping Others

Unlike \$project (which requires explicitly listing every field you want to KEEP), \$addFields adds or overwrites specific fields while automatically preserving every other existing field untouched.

```

db.orders.aggregate([
  { $addFields: {
    discountedTotal: { $multiply: ["$totalAmount", 0.9] },
    orderYear: { $year: "$orderDate" }
  } }
  // ALL original fields (customerId, items, status, etc.) remain intact
  // -- only the two new computed fields are added
])

```

8.9 Pipeline Stage Order and Performance

Because each stage processes the OUTPUT of the previous one, stage ORDER has a major performance impact — this is the aggregation-framework equivalent of choosing the right index.

1. Put \$match as EARLY as possible (ideally first) — filtering out irrelevant documents before expensive stages like \$group or \$unwind reduces the volume every later stage has to process, and allows the early \$match to use an index.
2. Put \$sort before \$limit — as shown in Section 8.5, sorting AFTER limiting produces incorrect results.
3. Use \$project or \$addFields to DROP unneeded large fields early if a later stage doesn't need them, reducing the amount of data flowing through the rest of the pipeline.

- Place \$unwind carefully — it can dramatically multiply the document count flowing downstream, so filter with \$match both BEFORE unwinding (on parent-level fields) and AFTER unwinding (on the now-flattened array-element fields) wherever possible.

NOTE: MongoDB's aggregation engine does perform SOME automatic optimization (e.g., it can reorder certain \$match stages earlier internally), but relying on the optimizer instead of writing an efficient pipeline yourself is not a substitute for good pipeline design — exactly parallel to relying on the query planner instead of creating a proper index.

8.10 Full Aggregation Stage Quick-Reference

Stage	Purpose
\$match	Filter documents (same syntax as find())
\$group	Group documents and compute aggregate values per group
\$project	Reshape output: include/exclude/rename/compute fields
\$addFields	Add or overwrite fields while keeping all others intact
\$sort	Order documents by one or more fields
\$limit / \$skip	Restrict/paginate the number of documents flowing through
\$unwind	Deconstruct an array field into one document per element
\$lookup	Left-outer-join against another collection
\$count	Count the documents at this point in the pipeline
\$facet	Run MULTIPLE independent sub-pipelines in parallel on the same input, combining results

9. Real-World Aggregation Examples

9.1 E-Commerce: Monthly Revenue Report by Category

```
db.orders.aggregate([
  { $match: { status: "completed", orderDate: { $gte: ISODate("2026-01-01") } } },
  { $unwind: "$items" },
  { $group: {
    _id: { category: "$items.category", month: { $month: "$orderDate" } },
    revenue: { $sum: { $multiply: ["$items.price", "$items.qty"] } },
    unitsSold: { $sum: "$items.qty" }
  } },
  { $sort: { "_id.month": 1, revenue: -1 } }
])
```

9.2 Social Media: Top 5 Most-Liked Posts Per User

```
db.posts.aggregate([
  { $sort: { authorId: 1, likeCount: -1 } },
  { $group: {
    _id: "$authorId",
    topPosts: { $push: { title: "$title", likes: "$likeCount" } }
  } },
  { $project: {
    topPosts: { $slice: ["$topPosts", 5] } // keep only the top 5 per user
  } }
])
```

9.3 HR Analytics: Average Salary by Department, With Headcount

```
db.employees.aggregate([
  { $match: { active: true } },
  { $group: {
    _id: "$department",
    avgSalary: { $avg: "$salary" },
    headcount: { $sum: 1 },
    totalPayroll: { $sum: "$salary" }
  } },
  { $sort: { avgSalary: -1 } },
  { $project: {
    department: "$_id", _id: 0,
    avgSalary: { $round: ["$avgSalary", 2] },
    headcount: 1, totalPayroll: 1
  } }
])
```

9.4 Ride-Hailing: Driver Leaderboard With Customer Join

```
db.trips.aggregate([
  { $match: { status: "completed", completedAt: { $gte: ISODate("2026-08-01") } } },
  { $group: {
    _id: "$driverId",
    totalTrips: { $sum: 1 },
    totalEarnings: { $sum: "$fareAtCompletion" }
  } },
  { $sort: { totalEarnings: -1 } },
  { $limit: 10 },
  { $lookup: {
    from: "drivers", localField: "_id", foreignField: "_id", as: "driverInfo"
  } },
  { $unwind: "$driverInfo" },
  { $project: {
    driverName: "$driverInfo.name", totalTrips: 1, totalEarnings: 1, _id: 0
  } }
])
```

9.5 Multi-Metric Dashboard With \$facet

\$facet runs several independent aggregation sub-pipelines against the SAME input simultaneously, combining their separate results into one document — perfect for a single dashboard query needing several unrelated summary metrics at once, avoiding multiple separate round trips.

```
db.orders.aggregate([
  { $match: { status: "completed" } },
  { $facet: {
    byRegion: [
      { $group: { _id: "$region", total: { $sum: "$totalAmount" } } }
    ],
    byStatus: [
      { $group: { _id: "$status", count: { $sum: 1 } } }
    ],
    topOrders: [
      { $sort: { totalAmount: -1 } }, { $limit: 5 },
      { $project: { customerId: 1, totalAmount: 1, _id: 0 } }
    ]
  } }
])

// Result: ONE document with THREE independent result arrays --
// byRegion, byStatus, and topOrders -- computed from a SINGLE pipeline run
```

NOTE: \$facet is a genuinely powerful, production-relevant tool: instead of running three separate aggregate() calls (three round trips, three separate scans of the filtered data), a single \$facet call computes all three metrics from ONE pass over the \$match-filtered data — a real, measurable efficiency gain for dashboard-style backends.

10. Comic-Style Scenario: The Library Card Catalog & The Assembly Line

Two intuitive analogies, one for indexing and one for the aggregation pipeline — together explaining this unit's two big ideas.

10.1 Part One: The Library Card Catalog (Indexing)

PANEL 1 — A Library With No Card Catalog (No Index)

Visitor: "Do you have any books by 'Rowling'?"

Librarian: "Let me check... *walks down EVERY aisle, checking EVERY book, one at a time*... this might take a while."

 (This is a *COLLSCAN*. Without an index, finding anything requires checking every single document, no matter how large the library.)

PANEL 2 — The Library Builds a Card Catalog (Creating an Index)

Librarian: "Let's build a card catalog sorted by author's last name -- each card points directly to a specific shelf and position."

```
db.books.createIndex({ author: 1 })
```

 (Building the catalog takes some upfront work and ongoing upkeep every time a new book arrives -- exactly like an index's write cost.)

PANEL 3 — Now the Same Search Is Instant (IXSCAN)

Visitor: "Books by 'Rowling'?"

Librarian: *flips directly to the 'R' section of the catalog* "Aisle 7, Shelf 3 -- right this way."

 (This is an *IXSCAN*. The catalog (index) let the librarian jump directly to the answer instead of checking every book in the building.)

PANEL 4 — But a Catalog Sorted ONLY by Author Can't Help With 'Books From 2020'

Visitor: "What about books published in 2020, regardless of author?"

Librarian: "Sorry -- my catalog is sorted by AUTHOR first. To answer that, I'd need a catalog sorted by YEAR, or a compound catalog sorted by (year, author) together."

 (This is the compound index *PREFIX RULE* from Section 3 -- a catalog's sort order determines EXACTLY which questions it can answer efficiently.)

10.2 Part Two: The Assembly Line (Aggregation Pipeline)

PANEL 5 — Raw Materials Enter the Factory (\$match)

Foreman: "First station: reject anything that isn't a 'completed' order -- no point processing cancelled ones further down the line."

```
{ $match: { status: "completed" } }
```

 (Filtering *FIRST*, before expensive processing, is why \$match belongs as early as possible in the pipeline -- Section 8.9.)

PANEL 6 — Items Are Grouped and Totaled (\$group)

Foreman: "Second station: sort every order into bins by customer, and total up each bin's value as it fills."

```
{ $group: { _id: "$customerId", total: { $sum: "$amount" } } }
```

 (Each 'bin' is a group; the accumulator (\$sum) tallies as documents flow through -- this station TRANSFORMS the shape of the data, from many small orders into few summarized totals.)

PANEL 7 — The Finished Product Rolls Off the Line (\$sort + \$limit)

Foreman: "Final station: arrange the finished bins from highest total to lowest, and package only the top 10 for shipping."

```
{ $sort: { total: -1 } }, { $limit: 10 }
```

 (Just like the real assembly line, *ORDER* matters -- sorting AFTER limiting would ship the wrong 10 bins entirely.)

11. Big Unified Example: Sales Analytics Dashboard (End-to-End)

This running example combines EVERY concept from this unit — indexing strategy, explain() diagnostics, and a multi-stage aggregation pipeline — into one realistic backend task: building the data layer for a sales analytics dashboard.

Step 1 — Seed a Realistic Dataset

```
use sales_analytics

const regions = ["North", "South", "East", "West"]
const statuses = ["completed", "cancelled", "pending"]
let salesBatch = []
for (let i = 1; i <= 20000; i++) {
  salesBatch.push({
    orderId: "ORD" + i,
    region: regions[i % regions.length],
    status: statuses[i % statuses.length],
    amount: 100 + (i % 5000),
    orderDate: new Date(2026, i % 12, (i % 28) + 1)
  })
}
db.sales.insertMany(salesBatch)
db.sales.countDocuments() // -> 20000
```

Step 2 — Baseline: Diagnose the Dashboard's Core Query Without an Index

```
// The dashboard's most frequent query: completed orders in a region, sorted by date
db.sales.find({ status: "completed", region: "North" })
  .sort({ orderDate: -1 }).explain("executionStats")
// winningPlan.stage: "COLLSCAN"
// totalDocsExamined: 20000 (the entire collection!)
// executionTimeMillis: (noticeably higher than it should be)
```

Step 3 — Apply the ESR Rule and Create the Right Index

Classifying the query's fields: status and region are both EQUALITY filters, orderDate is the SORT field. Following ESR (Section 3.3), equality fields lead, sort field follows.

```
db.sales.createIndex({ status: 1, region: 1, orderDate: -1 })

// Re-run the SAME query
db.sales.find({ status: "completed", region: "North" })
  .sort({ orderDate: -1 }).explain("executionStats")
// winningPlan.stage: "IXSCAN" (no separate in-memory SORT stage)
// totalDocsExamined: matches nReturned closely
```

```
// executionTimeMillis: dramatically lower
```

Step 4 — Build the Dashboard's Multi-Metric Aggregation With \$facet

```
db.sales.aggregate([
  { $match: { status: "completed" } }, // uses the index just created
  { $facet: {
    revenueByRegion: [
      { $group: { _id: "$region", total: { $sum: "$amount" } } },
      { $sort: { total: -1 } }
    ],
    monthlyTrend: [
      { $group: { _id: { $month: "$orderDate" }, total: { $sum: "$amount" } } },
      { $sort: { "_id": 1 } }
    ],
    topOrders: [
      { $sort: { amount: -1 } }, { $limit: 5 },
      { $project: { orderId: 1, amount: 1, region: 1, _id: 0 } }
    ],
    overallStats: [
      { $group: {
        _id: null, grandTotal: { $sum: "$amount" },
        avgOrderValue: { $avg: "$amount" }, totalOrders: { $sum: 1 }
      } }
    ]
  } }
])
```

NOTE: This single \$facet call computes FOUR independent dashboard widgets (revenue by region, monthly trend, top orders, overall stats) from ONE pass over the index-accelerated \$match-filtered data — combining Section 6's indexing discipline with Section 8's pipeline design into one realistic, production-grade query.

Step 5 — Verify the Aggregation Itself Uses the Index

```
// explain() works on aggregate() pipelines too, not just find()
db.sales.aggregate([
  { $match: { status: "completed", region: "North" } },
  { $sort: { orderDate: -1 } }
], { explain: true })

// Look for an IXSCAN as the input to the initial $cursor stage --
// confirming the $match (and even the $sort) is being served by
// the compound index created in Step 3, not a full collection scan
```

NOTE: End-to-end recap: Steps 1–3 apply Sections 1, 3, 6, and 7 (seed data, diagnose with explain(), apply ESR, verify the fix). Steps 4–5 apply Section 8 and 9 (a production-realistic multi-metric aggregation, verified with explain() on the pipeline itself). This mirrors exactly how a real backend engineer would approach building an analytics feature: get the indexing right FIRST, then build the aggregation on top of that solid foundation.

12. Full End-to-End Practical Walkthroughs

Two complete scenarios, each starting from opening the terminal through indexing strategy and a full aggregation pipeline — combining everything from Units I through IV into single, continuous, copy-pasteable sessions.

12.1 Walkthrough 1 — Restaurant Review Platform (Text Search + Aggregation)

Scenario: A restaurant review site needs full-text search over reviews AND a ratings-summary dashboard per restaurant.

Step 0 — Open Terminal, Start MongoDB, Connect

```
sudo systemctl start mongod
mongosh
```

Step 1 — Create Collections and Seed Data

```
use restaurant_reviews

db.restaurants.insertMany([
  { _id: "r1", name: "Spice Route", cuisine: "Indian", city: "Chandigarh" },
  { _id: "r2", name: "Pasta Palace", cuisine: "Italian", city: "Ludhiana" }
])

db.reviews.insertMany([
  { restaurantId: "r1", rating: 5, text: "Amazing butter chicken, best in the city!" },
  { restaurantId: "r1", rating: 4, text: "Great flavors but service was a bit slow." },
  { restaurantId: "r2", rating: 3, text: "Decent pasta, nothing special." },
  { restaurantId: "r2", rating: 5, text: "Best Italian food outside of Italy, incredible!" }
])
```

Step 2 — Create a Text Index and Search Reviews

```
db.reviews.createIndex({ text: "text" })

db.reviews.find({ $text: { $search: "best" } })

// With relevance score
db.reviews.find(
  { $text: { $search: "best" } },
  { score: { $meta: "textScore" }, text: 1, _id: 0 }
).sort({ score: { $meta: "textScore" } })
```

Step 3 — Aggregation: Average Rating Per Restaurant, Joined With Restaurant Info

```

db.reviews.aggregate([
  { $group: {
    _id: "$restaurantId",
    avgRating: { $avg: "$rating" },
    reviewCount: { $sum: 1 }
  } },
  { $lookup: {
    from: "restaurants", localField: "_id", foreignField: "_id", as: "info"
  } },
  { $unwind: "$info" },
  { $project: {
    _id: 0, name: "$info.name", cuisine: "$info.cuisine",
    avgRating: { $round: ["$avgRating", 1] }, reviewCount: 1
  } },
  { $sort: { avgRating: -1 } }
])

```

Step 4 — Index the Lookup Fields for Performance

```

// The $lookup above joins on restaurantId -- index it on the reviews side
// (the 'many' side of a One-to-Many relationship, Unit III Section 4.2.2)
db.reviews.createIndex({ restaurantId: 1 })

db.reviews.find({ restaurantId: "r1" }).explain("executionStats")
// winningPlan.stage: "IXSCAN"

```

NOTE: Recap: this walkthrough combined a text index (Section 4) for search, a compound aggregation pipeline with \$group + \$lookup + \$project (Section 8), and a supporting index on the join field — the exact combination a real review-platform backend would need.

12.2 Walkthrough 2 — IoT Sensor Dashboard (Compound Indexing + Time-Bucketed Aggregation)

Scenario: An IoT platform ingesting temperature readings needs both fast range queries by time AND an hourly-average aggregation report.

Step 0 — Open Terminal, Connect via Atlas

```
mongosh "mongodb+srv://<cluster-name>.mongodb.net/" --apiVersion 1 --username
<db_username>
```

Step 1 — Seed Sensor Readings

```
use iot_platform

let readings = []
for (let h = 0; h < 24; h++) {
  for (let m = 0; m < 60; m += 5) {
    readings.push({
      sensorId: "temp01",
      timestamp: new Date(2026, 7, 20, h, m),
      value: 20 + Math.random() * 10
    })
  }
}
db.readings.insertMany(readings)
db.readings.countDocuments() // -> 288
```

Step 2 — Diagnose the Common Range Query Without an Index

```
db.readings.find({
  sensorId: "temp01",
  timestamp: { $gte: new Date(2026,7,20,8,0), $lte: new Date(2026,7,20,10,0) }
}).explain("executionStats")
// winningPlan.stage: "COLLSCAN"
```

Step 3 — Apply ESR: sensorId (Equality) Then Timestamp (Range)

```
db.readings.createIndex({ sensorId: 1, timestamp: 1 })

db.readings.find({
  sensorId: "temp01",
  timestamp: { $gte: new Date(2026,7,20,8,0), $lte: new Date(2026,7,20,10,0) }
}).explain("executionStats")
// winningPlan.stage: "IXSCAN"
```

Step 4 — Aggregation: Hourly Average (Bucket Pattern in Action)

This directly reuses the Bucket Pattern concept from Unit III — here computed on the fly via aggregation rather than pre-bucketed at write time.

```
db.readings.aggregate([
  { $match: { sensorId: "temp01" } },
  { $group: {
    _id: { $hour: "$timestamp" },
    avgTemp: { $avg: "$value" },
    minTemp: { $min: "$value" },
    maxTemp: { $max: "$value" },
    readingCount: { $sum: 1 }
  } },
  { $sort: { "_id": 1 } },
  { $project: {
    hour: "$_id", _id: 0,
    avgTemp: { $round: ["$avgTemp", 2] }, minTemp: 1, maxTemp: 1, readingCount: 1
  } }
])
```

Step 5 — Verify Aggregation Performance With explain()

```
db.readings.aggregate([
  { $match: { sensorId: "temp01" } },
  { $group: { _id: { $hour: "$timestamp" }, avgTemp: { $avg: "$value" } } }
], { explain: true })
// Confirm the initial $match is served by IXSCAN on { sensorId: 1, timestamp: 1 }
```

NOTE: Recap: this walkthrough paired a compound index following ESR (Section 3) with a time-based \$group aggregation (Section 8) — precisely the pattern behind real IoT/monitoring dashboards, and a direct practical echo of Unit III's Bucket Pattern applied at query time instead of write time.

13. Practical Exercises (with Solutions)

Exercise 1 — Diagnose and Fix a Slow Query (Basic–Intermediate)

Task: Given a students collection with 100,000 documents, a query filtering { branch: "CSE", cgpa: { \$gt: 8 } } sorted by name is slow. Diagnose with explain(), classify the fields using ESR, create the correct index, and verify the fix.

Solution:

```
// Baseline
db.students.find({ branch: "CSE", cgpa: { $gt: 8 } })
  .sort({ name: 1 }).explain("executionStats")
// stage: COLLSCAN

// ESR classification: branch = Equality, name = Sort, cgpa = Range
db.students.createIndex({ branch: 1, name: 1, cgpa: 1 })

// Verify
db.students.find({ branch: "CSE", cgpa: { $gt: 8 } })
  .sort({ name: 1 }).explain("executionStats")
// stage: IXSCAN, no separate SORT stage
```

Exercise 2 — Build a Covered Query (Intermediate)

Task: On a products collection, design a covered query that returns only name and price for products in a given category, without ever fetching the full document.

Solution:

```
db.products.createIndex({ category: 1, name: 1, price: 1 })

db.products.find(
  { category: "Electronics" },
  { name: 1, price: 1, _id: 0 } // _id MUST be excluded for a covered query
).explain("executionStats")
// totalDocsExamined: 0 -- confirms this is a fully covered query
```

Exercise 3 — Multi-Stage Aggregation Report (Advanced)

Task: Using an orders collection (customerId, items: [{product, price, qty}], status, orderDate), write a pipeline that returns, for each customer with at least 3 completed orders: their total spend, order count, and their single most expensive item purchased — sorted by total spend descending.

Solution:

```

db.orders.aggregate([
  { $match: { status: "completed" } },
  { $unwind: "$items" },
  { $group: {
    _id: "$customerId",
    totalSpend: { $sum: { $multiply: ["$items.price", "$items.qty"] } },
    orderCount: { $addToSet: "$_id" }, // collect unique order ids seen
    maxItemPrice: { $max: "$items.price" }
  } },
  { $project: {
    totalSpend: 1, maxItemPrice: 1,
    orderCount: { $size: "$orderCount" }
  } },
  { $match: { orderCount: { $gte: 3 } } },
  { $sort: { totalSpend: -1 } }
])

```

NOTE: Notice the SECOND \$match after \$group — filtering on an AGGREGATED value (orderCount) can only happen AFTER the \$group stage computes it, unlike the first \$match which filters raw input documents before any transformation. This two-\$match pattern (pre-filter raw data, post-filter aggregated results) is extremely common in real reporting pipelines.

14. Self-Assessment: Conceptual Review Questions (with Model Answers)

Question 1

A compound index is created as `{ status: 1, region: 1, orderDate: -1 }`. Explain which of the following queries can use this index efficiently, and why: (a) `find({status: 'shipped'})`, (b) `find({region: 'North'})`, (c) `find({status:'shipped', region:'North'}).sort({orderDate:-1})`.

ANSWER: Model Answer:

(a) CAN use the index efficiently — status is the LEADING field of the compound index, satisfying the prefix rule (Section 3.2) on its own. (b) CANNOT use this index efficiently — region is not the leading field, so a query filtering on region alone doesn't match a valid prefix of the index and falls back to a collection scan. (c) CAN use the index very efficiently — status and region are both equality filters matching the index's first two fields in order, and orderDate is the sort field matching the index's third field, meaning MongoDB can serve BOTH the filter and the sort directly from the index with no separate in-memory sort step.

Question 2

Explain the difference between `totalDocsExamined` and `totalKeysExamined` in an `explain('executionStats')` output, and describe what it means when `totalDocsExamined` is 0.

ANSWER: Model Answer:

`totalKeysExamined` counts how many entries in an index's B-tree were scanned to find matching documents. `totalDocsExamined` counts how many ACTUAL documents were fetched from the collection's storage to satisfy the query. When `totalDocsExamined` is 0, it means MongoDB never needed to fetch any actual document data at all — every field required by the query (both filter and projection) was already present within the index itself. This is called a COVERED QUERY (Section 7.1), the fastest possible category of MongoDB query, since it avoids the cost of fetching full documents from disk/cache entirely.

Question 3

A developer writes an aggregation pipeline that places `$sort` AFTER `$limit`, intending to return the top 10 highest-value orders. Explain why this produces incorrect results, and provide the corrected pipeline.

ANSWER: Model Answer:

Placing `$limit` before `$sort` takes an ARBITRARY 10 documents from the collection first (in whatever order they happened to be encountered), and only THEN sorts those 10 — meaning the result is 'some 10 orders, sorted,' not 'the actual top 10 highest-value orders.' The pipeline stages execute strictly in the order written, each stage operating only on the output of the previous one, so an early `$limit` permanently discards documents before `$sort` ever gets a chance to consider them.

Corrected pipeline: `[ { $sort: { amount: -1 } }, { $limit: 10 } ]` — sorting ALL matching documents by amount descending FIRST, and only THEN taking the top 10 from that correctly-ordered full result set. This mirrors the identical ordering rule for `find()` cursor methods established in Unit II (sort before limit).

15. Industry Interview & Viva Question Bank (32 Questions)

Strictly scoped to Unit IV syllabus content, following the definition-first, scoring-oriented format established in previous units.

Q1. What is a MongoDB index, and what internal data structure implements it?

SCORING ANSWER: An index is a separate, ordered data structure that maps field values to document locations, allowing MongoDB to find matching documents without scanning the entire collection. It is implemented internally as a B-tree.

Q2. What is a COLLSCAN, and when does MongoDB perform one?

SCORING ANSWER: A COLLSCAN (collection scan) is when MongoDB examines every document in a collection one by one to find matches, because no usable index exists for the query's filter. It's the default, slowest fallback behavior.

Q3. Which index does every MongoDB collection have automatically, with zero manual setup?

SCORING ANSWER: An index on the `_id` field, created automatically the moment the collection is created — this is why queries filtering by `_id` are always efficient even with no other indexes present.

Q4. What does creating an index with `{ unique: true }` enforce?

SCORING ANSWER: It rejects any insert or update that would create a duplicate value for the indexed field, transforming the index from a pure performance tool into a data-integrity constraint, similar to a SQL UNIQUE constraint.

Q5. Explain the difference between a sparse index and a partial index.

SCORING ANSWER: A sparse index includes only documents where the indexed field EXISTS. A partial index includes only documents matching an arbitrary filter expression you define — a strict superset of sparse behavior, since 'field exists' is just one possible filter condition among many.

Q6. What type of field must a TTL index be created on, and can it be part of a compound index?

SCORING ANSWER: A TTL index must be a single-field index on a Date (or array-of-Date) field, specified with `expireAfterSeconds`. It cannot be part of a compound index — it must stand alone.

Q7. Explain the prefix rule for compound indexes with an example.

SCORING ANSWER: A compound index on `{a:1, b:1, c:1}` can efficiently support queries filtering on prefixes: `{a}`, `{a,b}`, or `{a,b,c}` — but NOT on `{b}` alone or `{c}` alone, since those don't match a leading, contiguous prefix of the indexed fields.

Q8. State the ESR rule and explain why Range fields come last.

SCORING ANSWER: ESR stands for Equality fields first, Sort fields second, Range fields last. Range fields come last because a range condition scans a CONTIGUOUS SPAN of the B-tree rather than a single point; placing a range field before a sort field would break the index's ability to also deliver results in pre-sorted order.

Q9. Why can a compound index include at most one array field?

SCORING ANSWER: Because indexing two array fields together would require an index entry for EVERY combination of elements from both arrays (a combinatorial explosion), which MongoDB disallows outright — attempting to create such an index raises a 'cannot index parallel arrays' error.

Q10. What is a multikey index, and is it something you configure explicitly?

SCORING ANSWER: A multikey index is automatically created when you index a field that holds an array — MongoDB creates one index entry per array element, all pointing to the same parent document. It's detected and applied automatically; you don't request it explicitly.

Q11. When is a hashed index preferred over a normal index, and what capability does it sacrifice?

SCORING ANSWER: Hashed indexes are preferred when you need an EVEN, unpredictable distribution of values — most commonly as a shard key for horizontal scaling. They sacrifice range-query support, since hashing destroys the original ordering of values.

Q12. What is a wildcard index, and what Unit III design pattern does it directly support?

SCORING ANSWER: A wildcard index (using the `$**` syntax) indexes an unknown or highly variable set of fields across documents, useful for schema-flexible collections. It directly supports the Attribute Pattern from Unit III, allowing queries on any `{k,v}` attribute without knowing attribute names in advance.

Q13. A collection can have how many text indexes at most?

SCORING ANSWER: At most ONE text index per collection — though that single text index can span multiple fields simultaneously.

Q14. What does the weights option do when creating a multi-field text index?

SCORING ANSWER: It assigns relative importance to matches in different fields when computing a relevance score — e.g., a match in a 'title' field can be weighted more heavily than a match in a 'content' field, so title matches rank higher in `$meta: 'textScore'` results.

Q15. Define selectivity, and explain why a 'status' field with only two possible values is usually a poor standalone index candidate.

SCORING ANSWER: Selectivity measures how effectively matching a field's value narrows the search — a highly selective field has many distinct values, so a match eliminates most other documents. A 'status' field with only 2 values has very low selectivity: matching one value might still return a huge fraction of the entire collection, so an index on it alone provides little benefit over a collection scan, despite still incurring the full write-cost of maintaining it.

Q16. Distinguish cardinality from selectivity.

SCORING ANSWER: Cardinality is a property of the FIELD — the raw count of distinct values it has across the collection. Selectivity is a property of a specific QUERY VALUE — what fraction of documents that particular value matches. A field can have high overall cardinality while a specific value still has low selectivity (e.g., 'city' has many distinct values overall, but querying a very common city could still match a large fraction of documents).

Q17. What is index intersection, and why is a purpose-built compound index usually preferred over relying on it?

SCORING ANSWER: Index intersection is when MongoDB's query planner combines results from two SEPARATE single-field indexes to satisfy one query. It's generally less efficient than a single, deliberately-designed compound index covering the same query, so it should be understood as a planner fallback, not a design strategy to rely on.

Q18. What is a covered query, and what must be true of the projection for a query to be covered?

SCORING ANSWER: A covered query is one where every field in both the filter AND the projection is present within the index itself, so MongoDB never fetches the actual document — answering entirely from the index. The projection must explicitly exclude `_id` (`_id: 0`) unless `_id` itself is part of the index, since `_id` is returned by default otherwise, forcing a document fetch.

Q19. List the three verbosity modes of `explain()` and what each one shows.

SCORING ANSWER: 'queryPlanner' (default) shows only the chosen and rejected candidate plans without running the query. 'executionStats' actually runs the query and reports real timing, docs examined, and keys examined. 'allPlansExecution' runs the query AND all candidate plans considered, with stats for each — used for deep debugging of the planner's decision.

Q20. In an executionStats output, what does a large gap between `totalDocsExamined` and `nReturned` indicate?

SCORING ANSWER: It indicates an inefficient query — MongoDB examined far more documents than it actually needed to return, almost always signaling a missing or poorly-ordered index for that query's filter.

Q21. What is the purpose of the Aggregation Pipeline, and how does it conceptually differ from `find()`?

SCORING ANSWER: The Aggregation Pipeline processes documents through a sequence of transformation STAGES, each taking the previous stage's output as input, enabling data TRANSFORMATION and SUMMARIZATION (totals, averages, grouped reports) that `find()` cannot express — `find()` is designed for retrieving documents largely as-is, not summarizing across many documents.

Q22. Why should `$match` generally be the first stage in an aggregation pipeline?

SCORING ANSWER: Because filtering out irrelevant documents as early as possible reduces the volume of data every subsequent, potentially expensive stage (like `$group` or `$unwind`) has to process, and critically, an early `$match` can use an index exactly like a normal `find()` query — later stages cannot.

Q23. Explain what `$group` with `_id: null` accomplishes.

SCORING ANSWER: It places every document flowing into the stage into a SINGLE group (since all documents share the same null grouping key), letting you compute a grand total or overall summary statistic across the entire filtered dataset in one accumulator operation.

Q24. What does `$unwind` do, and why is it necessary before aggregating on individual array elements?

SCORING ANSWER: `$unwind` deconstructs an array field, producing one separate output document for EACH element of the array. It's necessary because `$group` and other stages operate on whole documents — without unwinding, you cannot aggregate metrics per INDIVIDUAL array element (e.g., total revenue per product across many orders' embedded item arrays), only per whole document.

Q25. Why must `$sort` come before `$limit` in an aggregation pipeline intending to return the 'top N' results?

SCORING ANSWER: Pipeline stages execute strictly in written order, each operating only on the previous stage's output. Placing `$limit` before `$sort` would select an arbitrary N documents first (in whatever order encountered) and only then sort those N — producing 'some N results, sorted' rather than the true top N by the intended criterion.

Q26. What does `$addFields` do differently from `$project`?

SCORING ANSWER: `$addFields` adds or overwrites specific fields while automatically preserving every other existing field untouched. `$project` requires explicitly listing every field you want to KEEP (an inclusion projection) or exclude, making it easy to accidentally drop fields you didn't intend to remove.

Q27. Describe what `$facet` does and give a scenario where it provides a real efficiency benefit.

SCORING ANSWER: `$facet` runs multiple independent aggregation sub-pipelines against the SAME input simultaneously, combining their separate results into one output document. It benefits a dashboard needing several unrelated summary metrics (e.g., revenue by region, monthly trend, top orders) computed from one `$match`-filtered dataset in a SINGLE pass, avoiding multiple separate round trips and re-scans.

Q28. In a pipeline with two separate `$match` stages (one before `$group`, one after), what is the functional difference between them?

SCORING ANSWER: The `$match` BEFORE `$group` filters raw INPUT documents before any transformation, and can use an index. The `$match` AFTER `$group` filters the AGGREGATED, computed output of the `$group` stage (e.g., filtering on a computed count or sum) — this can only happen after `$group` produces those values, and cannot use a collection index since it's operating on already-transformed data.

Q29. What syntax distinguishes a basic `$lookup` from an advanced, pipeline-based `$lookup`?

SCORING ANSWER: A basic `$lookup` uses `localField/foreignField` for simple equality-based joining. An advanced `$lookup` instead uses a `let` block to define variables from the local document, plus a full pipeline array (which can include `$match` with `$expr`, `$project`, and other stages) to filter and shape the joined data before it's attached — enabling more complex join conditions than simple field equality.

Q30. How would you verify whether an aggregation pipeline's initial stages are using an index?

SCORING ANSWER: By running the aggregation with the `explain` option (e.g., `db.collection.aggregate([...], {explain: true})`) and inspecting the resulting execution plan for an `IXSCAN` stage feeding into the pipeline's initial `$cursor` stage — confirming the `$match` (and sometimes `$sort`) is being served by an index rather than a full collection scan.

Q31. Why is index intersection generally NOT a substitute for good compound index design, even though MongoDB supports it?

SCORING ANSWER: Index intersection requires the planner to scan and merge results from multiple separate index structures at query time, which carries more overhead than traversing a single, purpose-built compound index designed around the query's actual equality/sort/range fields (following ESR). It should be treated as an occasional planner fallback, not a deliberate design strategy.

Q32. Summarize, in one sentence, the professional workflow for diagnosing and fixing a slow MongoDB query.

SCORING ANSWER: Run `explain('executionStats')` to identify a COLLSCAN or a large `totalDocsExamined`-vs-`nReturned` gap, classify the query's fields as Equality/Sort/Range, create a compound index following the ESR rule, and re-run `explain()` to confirm an IXSCAN with `totalDocsExamined` now closely matching `nReturned`.

STUDY TIP: Exam/viva technique: Unit IV examiners love asking you to trace an `explain()` output or an aggregation pipeline STEP BY STEP and predict the result or diagnose the problem. Practice reading real execution plans and pipelines line by line rather than only memorizing definitions — the applied skill is what's actually being tested.

16. Frequently Confused Concepts — Quick Clarifications

A COLLSCAN vs. an IXSCAN

A COLLSCAN examines every document in the collection sequentially because no usable index exists. An IXSCAN uses an index's B-tree to jump directly to matching entries, examining far fewer documents — the key distinction to look for in every `explain()` output.

Sparse index vs. Partial index

A sparse index is a narrow, single-purpose tool that only includes documents where a field EXISTS. A partial index is strictly more general, including documents matching ANY filter expression you define — sparse behavior can be reproduced with a partial index, but not vice versa.

The prefix rule vs. the ESR rule

The prefix rule describes WHICH field COMBINATIONS a compound index can serve (any leading, contiguous subset of its fields). The ESR rule describes HOW to ORDER those fields in the first place (Equality, then Sort, then Range) so the index serves your actual query patterns as efficiently as possible — ESR is how you DESIGN the index; the prefix rule is how you PREDICT what it can serve afterward.

totalDocsExamined vs. totalKeysExamined

`totalKeysExamined` counts index B-tree ENTRIES scanned. `totalDocsExamined` counts actual DOCUMENTS fetched from storage. A covered query has `totalDocsExamined` = 0 while `totalKeysExamined` is still equal to `nReturned`, since the index entries themselves already contained everything needed.

A covered query vs. an ordinary indexed query

An ordinary indexed query uses an index to LOCATE matching documents efficiently, but still fetches each full document afterward (a FETCH stage follows the IXSCAN). A covered query answers the ENTIRE request from the index alone, never fetching the document — strictly faster, but only possible when every needed field is part of the index.

\$match vs. \$project

`$match` FILTERS which documents continue through the pipeline (keeping or discarding whole documents). `$project` RESHAPES each surviving document's fields (adding, removing, renaming, or computing fields) without discarding any documents based on their content.

\$project vs. \$addFields

`$project` requires explicitly listing every field to KEEP (or exclude) — anything not listed for inclusion is dropped. `$addFields` adds/overwrites specific named fields while automatically preserving every other existing field, making it the safer choice when you only want to ADD a computed field without restating the whole document shape.

\$group's accumulator \$push vs. \$addToSet

\$push collects EVERY value encountered into an array, including duplicates. \$addToSet collects only UNIQUE values, silently skipping duplicates — otherwise functionally identical.

Index intersection vs. a compound index

Index intersection combines results from two SEPARATE single-field indexes at query time as a planner fallback. A compound index is a single, purpose-built structure covering multiple fields together, deliberately designed via the ESR rule — generally more efficient and the professional default over relying on intersection.

Selectivity vs. Cardinality

Cardinality is the raw count of DISTINCT VALUES a field has across the whole collection (a property of the field). Selectivity is how much a SPECIFIC query value narrows the result set relative to the collection size (a property of a particular query against that field) — high cardinality often, but not always, implies high selectivity for any given value.

End of Unit IV. When you're ready, say "start Unit 5" to continue into MongoDB Integration with Node.js and Python.